

09. 추천 시스템

✓ 01. 추천 시스템의 개요와 배경

📌 추천 시스템이란?

- 사용자 취향을 파악해 맞춤형 콘텐츠나 상품을 자동으로 추천하는 시스템.
- 전자상거래(아마존, 이베이), 콘텐츠 플랫폼(유튜브, 넷플릭스, 애플뮤직) 등에서 핵심 기술로 사용됨.

📌 중요한 이유

- 매출 증가: 아마존 등은 추천 시스템 도입 후 판매량 급증.
- 사용자 경험 향상: 나도 몰랐던 취향까지 추천받아 더 오래 머무르게 됨.
- 신뢰도 상승: 좋은 추천은 사이트에 대한 신뢰를 높이고 재방문을 유도.
- 데이터의 선순환: 추천 → 클릭 → 데이터 축적 → 더 정밀한 추천.

📌 온라인에서의 필수 요소

- 상품이 많고, 메뉴 구성 복잡한 온라인 환경에서는 탐색 비용이 큼.
- 추천 시스템은 이런 복잡함을 줄여 쇼핑 스트레스 완화 + 선택 지원 역할.
- 예: "당신만을 위한 추천", "이 상품을 본 사람은 이런 것도 봤어요"

📌 추천 시스템이 사용하는 데이터

데이터 유형	예시
행동 로그	클릭, 조회, 구매, 찜, 장바구니 등
평가 정보	평점, 리뷰 내용
명시적 취향	선호 장르 선택 등
유사 사용자 행동	다른 사람의 반응 패턴

📌 추천 시스템의 주요 방식

(1) 콘텐츠 기반 필터링 (Content-Based Filtering)

- 사용자의 과거 관심 아이템과 비슷한 속성을 가진 상품 추천
- 예: "이전에 본 영화와 유사한 장르의 영화 추천"

- 사용자의 개인적 선호도 반영이 강함

(2) 협업 필터링 (Collaborative Filtering)

- 다른 사용자들과의 유사성 기반 추천
- 사용자가 아직 보지 않았지만 비슷한 사용자가 좋아한 것 추천

2-1. 최근접 이웃 기반 (User-based / Item-based)

- 유사한 사용자끼리 또는 아이템끼리 연결해서 추천
- 예: 아마존은 아이템 기반 최근접 이웃 사용

2-2. 잠재 요인 기반 (Latent Factor Model)

- 대표적으로 행렬 분해(Matrix Factorization) 기법 사용
- 사용자의 숨겨진 취향과 아이템의 특성을 벡터로 모델링
- 예: 넷플릭스 대회 우승 기법

(3) 하이브리드 필터링 (Hybrid)

- 콘텐츠 기반 + 협업 기반을 조합해 더 정밀하게 추천
- 다양한 데이터와 시나리오에 유연하게 대응 가능

✓ 02. 콘텐츠 기반 필터링 추천 시스템

📌 개념

- 사용자가 좋아한 아이템의 특징(콘텐츠)을 분석하여, 유사한 콘텐츠를 가진 다른 아이템을 추천하는 방식.

📌 작동 방식

- 추천 기준은 사용자 → 아이템 → 아이템의 속성입니다.
- 사용자가 높은 점수를 준 아이템들의 콘텐츠 정보(장르, 키워드, 작가 등)를 기반으로 유사한 아이템을 탐색합니다.

📌 예시: 영화 추천 시스템

가정: 사용자가 다음 두 영화에 높은 평점을 줌

- '컨택트' (8점) → 장르: SF, 미스터리 / 감독: 드니 빌뇌브
- '프로메테우스' (9점) → 장르: SF, 액션, 스릴러 / 감독: 리들리 스콧

→ 추천 대상:

- '블레이드 러너 2049'
 - 이유: SF 장르 포함
 - 드니 빌뇌브 감독
 - 리들리 스콧의 전작 리메이크
 - 사용자가 선호했던 여러 콘텐츠 특성을 공유함

✅ 03. 최근접 이웃 협업 필터링

📌 개념

- 사용자 행동(평점, 클릭, 구매 이력 등)을 기반으로, 비슷한 취향의 사용자들이 좋아한 아이템을 추천하는 방식
- 마치 친구에게 "이 영화 어때?" 하고 추천받는 것과 유사함

📌 작동 방식

과정	설명
입력	사용자-아이템 평점 데이터 (User-Item Matrix)
목표	사용자가 평가하지 않은 아이템의 평점 예측
핵심 로직	사용자 또는 아이템 간 유사도(similarity)를 계산하여 추천
결과	예측된 평점 또는 유사도 기반으로 추천 리스트 생성

📌 데이터 형태

로우 레벨 (원본 형태)

UserID	ItemID	Rating
User1	Item1	3
User2	Item2	1

사용자-아이템 행렬 (변환 형태)

	Item1	Item2	Item3
User1	3		3
User2	4	1	

- 대부분 희소 행렬(Sparse Matrix) 형태
- `pivot_table()` 등을 사용해 변환 가능

📌 유형별 정리

사용자 기반 협업 필터링 (User-based)

- 특정 사용자와 비슷한 사용자 Top-N 추출 → 그들이 좋아한 아이템을 추천
- 사용자의 평점 분포를 기반으로 사용자 간 유사도를 계산 (ex. 코사인 유사도)
- 예시:
 - 사용자 A와 B가 비슷한 취향 → B가 좋아한 '프로메테우스'를 A에게 추천

아이템 기반 협업 필터링 (Item-based)

- 특정 아이템과 유사한 아이템을 추천
- 사용자들의 평가 경향을 기반으로 아이템 간 유사도 계산
- 예시:
 - 사용자 D가 좋아한 '다크 나이트'와 유사한 평점을 받은 '프로메테우스'를 추천

📌 비교: 사용자 기반 vs. 아이템 기반

구분	사용자 기반	아이템 기반
기준	사용자 간 유사도	아이템 간 유사도
설명	나와 비슷한 유저가 좋아한 아이템 추천	내가 좋아한 아이템과 유사한 아이템 추천
정확도	낮은 경우도 있음 (데이터 희소성 등)	상대적으로 높음
실용성	계산 비용 큼, 확장 어려움	추천 정확도 높아 실제로 더 자주 사용됨

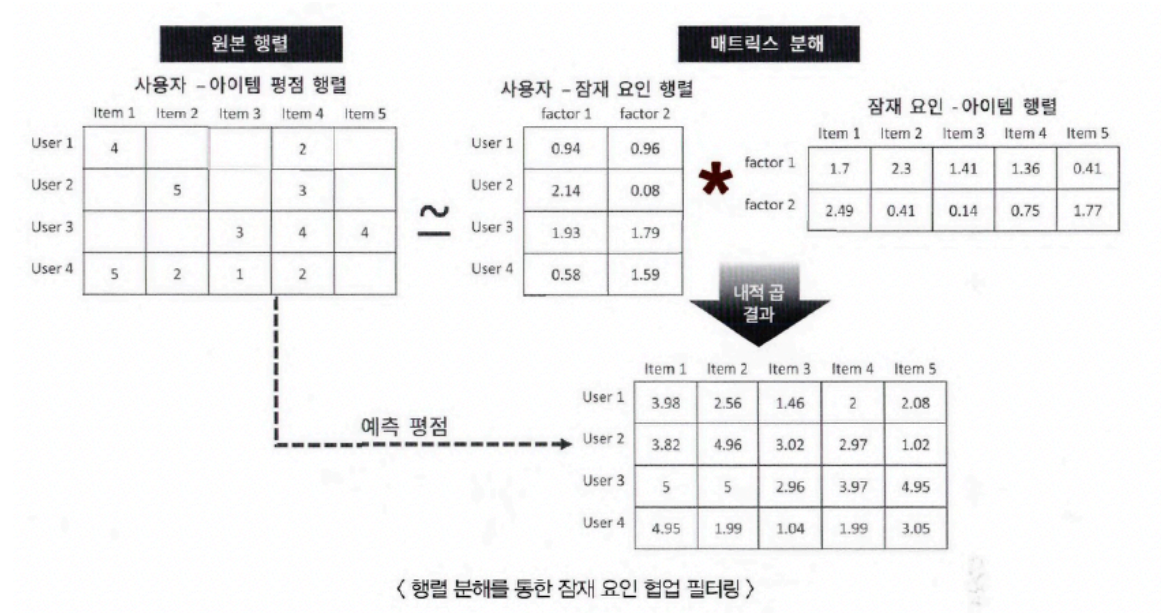
- 대부분의 추천 시스템은 아이템 기반 협업 필터링을 사용
- 코사인 유사도가 유사도 계산에 가장 많이 사용됨 (텍스트와 마찬가지로 벡터 간 각도를 이용)

✅ 04. 잠재 요인 협업 필터링

📌 개념

- 사용자-아이템 평점 행렬에 내재된 잠재 요인(latent factor) 을 찾아서, 사용자가 평가하지 않은 아이템의 평점을 예측하고 추천하는 방법.

- 행렬 분해(Matrix Factorization) 기법을 사용하여 평점 행렬을 저차원 형태로 분해함.

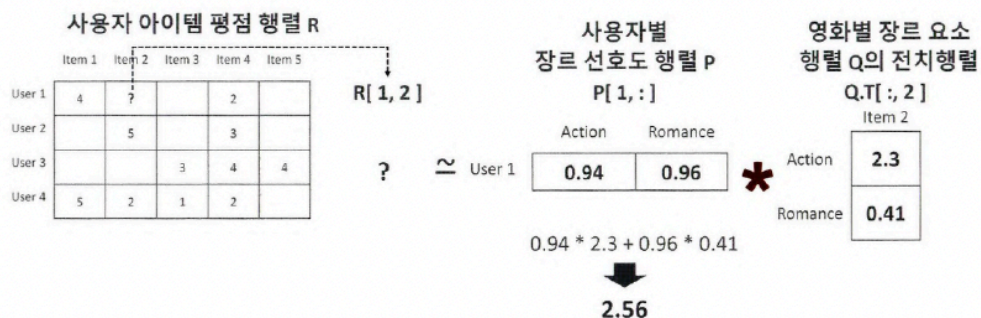


📌 핵심 아이디어

- R: 원본 사용자-아이템 평점 행렬 (크기: $M \times N$)
- P: 사용자-잠재 요인 행렬 (크기: $M \times K$)
- Q: 아이템-잠재 요인 행렬 (크기: $N \times K$)
- K: 잠재 요인의 수 (e.g., 액션 선호도, 로맨스 선호도 등)

📌 예시 (영화 추천)

- User 1은 액션과 로맨스 장르에 대한 선호도가 높음 (P 행렬).
- Item 1은 액션 요소가 강한 영화 (Q 행렬).
- 내적 계산을 통해 User 1이 Item 1에 줄 평점을 예측 가능.



장점

- 희소한 평점 행렬에서도 잠재적인 패턴을 추출할 수 있음.
- 사용자 취향과 아이템 특성을 숨어 있는 형태로 추정 가능.
- 예측 성능이 일반적인 협업 필터링보다 우수함.

행렬 분해란?

- 고차원 행렬을 두 개의 저차원 행렬로 나누는 기법
- 복잡한 다항식을 단순한 인수의 곱으로 나누는 "인수분해"처럼,
평점 행렬 R 을 두 개의 행렬(P, Q)로 나눔

목적

- 사용자-아이템 평점 행렬(R)에 존재하는 숨겨진 구조(잠재 요인)를 발견
- 예측하지 않은 평점을 추정하고 추천 시스템에 활용

주요 기법

기법	설명
SVD (Singular Value Decomposition)	고전적인 행렬 분해 기법. NaN 없는 행렬에만 적용 가능
NMF (Non-Negative Matrix Factorization)	모든 값이 0 이상인 행렬 분해
SGD (Stochastic Gradient Descent)	경사 하강법을 통해 최적의 행렬을 찾음
ALS (Alternating Least Squares)	SGD의 대안으로 교대로 최적화 수행

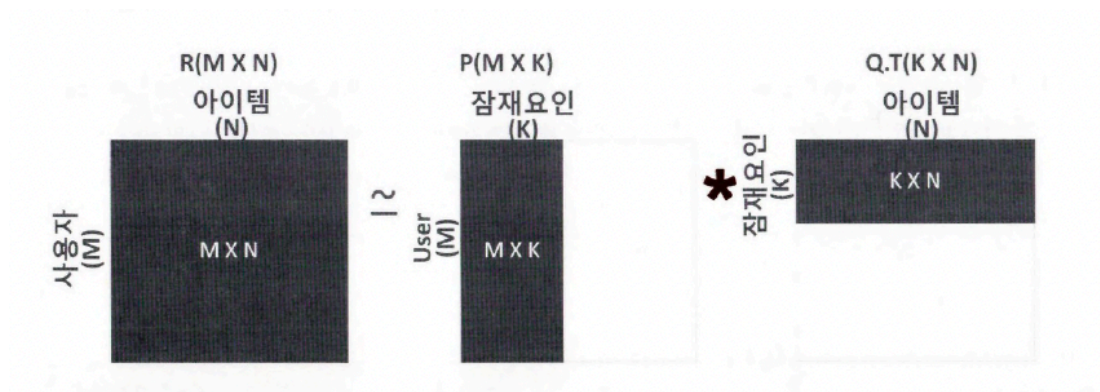
수학적 구조

R 행렬

- M 명의 사용자 $\times$ N 개의 아이템으로 구성된 원본 평점 행렬

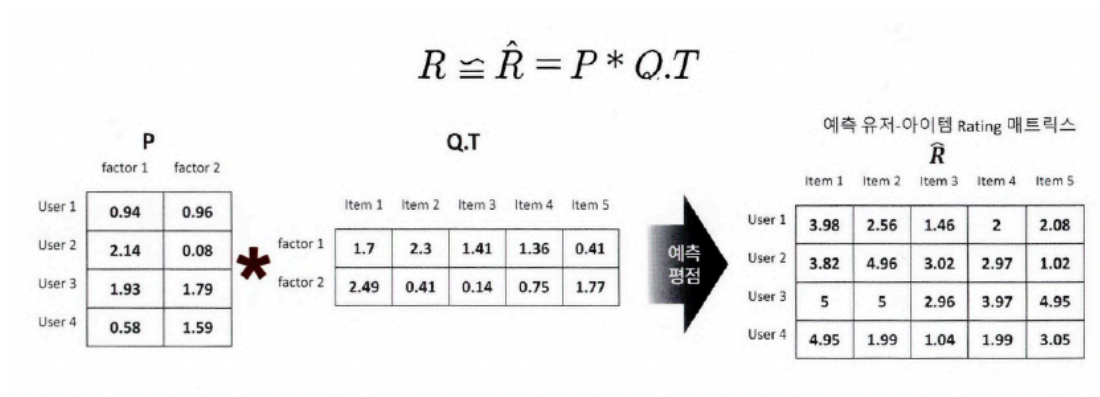
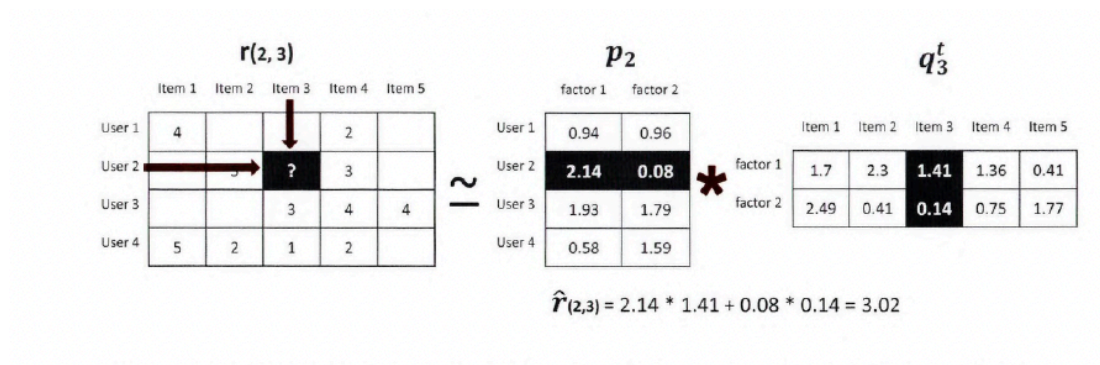
분해된 행렬

- 사용자-잠재 요인 행렬
- 아이템-잠재 요인 행렬
- 예측 행렬



📌 예시

- User 1이 액션을 좋아하고 Item 1이 액션 영화라면, 이 둘의 내적값은 높아지고 → 예측 평점도 높게 나옴



📌 SVD의 한계와 보완

- SVD는 결측값(NaN)이 있는 행렬에서는 사용 불가
- 이를 보완하기 위해 SGD나 ALS와 같은 최적화 기반 기법을 활용

📌 실제 예측 흐름

- 사용자-아이템 평점 행렬 R 생성 (NaN 포함 가능)

2. 임의로 초기화된 P, Q로 시작
3. 예측 평점 $\hat{R} = P \cdot Q^T$ 계산
4. 실제 평점과 비교해 오차 계산 (예: RMSE)
5. SGD 등을 이용하여 P, Q 값을 반복적으로 업데이트
6. 예측 행렬 완성

확률적 경사 하강법을 이용한 행렬 분해 개요

- SGD (Stochastic Gradient Descent)는 행렬 분해 시, 예측 평점과 실제 평점 간의 오차를 최소화하도록 사용자-아이템 행렬을 반복적으로 최적화하는 기법입니다.
- 사용자의 잠재 요인 벡터 PP, 아이템의 잠재 요인 벡터 QQ를 업데이트하며 최적의 추천 예측 모델을 만들어갑니다.
- 사용자-아이템 평점 행렬 RR을 사용자-잠재요인 행렬 PP와 아이템-잠재요인 행렬 QQ로 분해

비용 함수 (Loss Function)

예측값과 실제값의 차이 제곱 오차에 L2 정규화(Overfitting 방지)를 더한 것:

$$\min \sum (r_{u,i} - p_u q_i^t)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

$$p'_u = p_u + \eta (e_{(u,i)} * q_i - \lambda * p_u)$$

$$q'_i = q_i + \eta (e_{(u,i)} * p_u - \lambda * q_i)$$

SGD 학습 절차

1. 초기화: P와 Q를 정규분포 기반 랜덤값으로 초기화
2. 반복 학습:
 - 예측 평점 계산
 - 오차 계산
 - P, Q 업데이트

3. RMSE 평가 (선택):

예측 행렬과 실제 평점 간의 Root Mean Square Error 계산

✓ 08.파이썬 추천 시스템 패키지 - Surprise

- Surprise는 추천 시스템 전용 파이썬 라이브러리로, 다양한 협업 필터링 알고리즘을 간편하게 사용할 수 있도록 설계됨.
- 사이킷런과 유사한 API 구조를 제공하여 사용이 쉬움.
- 추천 시스템 구현, 모델 학습, 예측, 성능 평가, 하이퍼파라미터 튜닝 등을 지원.

📌 Surprise의 주요 특징

- 다양한 알고리즘 제공:
 - SVD, SVD++, NMF, KNNBasic, KNNBaseline, CoClustering 등
- 간단한 API:
 - `fit()`, `predict()`, `test()`, `train_test_split()`, `cross_validate()`, `GridSearchCV`
- 다양한 데이터 로딩 지원:
 - `load_builtin()` - MovieLens 내장 데이터
 - `load_from_file()` - 파일에서 로딩 (Reader 필요)
 - `load_from_df()` - 판다스 DataFrame에서 로딩

📌 데이터 구조 및 로딩

- 데이터 형식은 반드시 row-level: `user_id`, `item_id`, `rating` 순서의 3열 구성
- 내장 데이터 로딩

```
data = Dataset.load_builtin('ml-100k')
```

- CSV 파일 로딩

```
reader = Reader(line_format='user item rating timestamp', sep=',', rating_scale=(0.5, 5))
data = Dataset.load_from_file('ratings_noh.csv', reader=reader)
```

- DataFrame에서 로딩

```
data = Dataset.load_from_df(df[['userId', 'movieId', 'rating']], reader)
```

학습과 예측

1. 데이터 분할

```
trainset, testset = train_test_split(data, test_size=0.25)
```

2. 모델 학습 및 예측

```
algo = SVD(n_factors=50, random_state=0)
algo.fit(trainset)
predictions = algo.test(testset)
```

3. 예측 평점 확인

```
for pred in predictions[:3]:
    print(pred.uid, pred.iid, pred.est)
```

4. RMSE 평가

```
accuracy.rmse(predictions)
```

사용자 맞춤 추천

- 특정 사용자가 아직 보지 않은 영화 목록 추출
- `predict()` 를 통해 각 영화에 대한 예측 평점 계산
- 예측 평점 순으로 정렬하여 Top-N 영화 추천

교차 검증 및 튜닝

- 교차 검증

```
cross_validate(algo, data, measures=['RMSE', 'MAE'], cv=5)
```

- GridSearchCV 하이퍼파라미터 튜닝

```
param_grid = {'n_epochs': [20, 40], 'n_factors': [50, 100]}
gs = GridSearchCV(SVD, param_grid, measures=['rmse'], cv=3)
gs.fit(data)
```

주요 알고리즘 성능 (RMSE 기준)

알고리즘	RMSE	시간
SVD	0.934	11초
SVD++	0.920	9분
KNNBaseline	0.931	12초
CoClustering	0.963	3초

Baseline (편향 반영)

- 사용자/아이템 평점 경향을 반영하여 평점을 조정함

```
baseline_rating = 전체 평균 + 사용자 편향 + 아이템 편향
```